Linear-Gaussian State Space Models and the Kalman Filter

Victor Sze Lozko Pilev Nigel Prem Lim Aseem Kulkarni

Supervisor: Dr Sam Power

November 12, 2026

Abstract

To write here.

- Write this last.
- Four or five sentences: the problem (a hidden state seen only through noisy measurements), the linear-Gaussian case as the one setting solved exactly, what the report derives and implements, and what the simulation study shows.

Contents

1 Introduction and aims **2**
1.1 Why linear-Gaussian models? . . . 2

2 Model and tasks **3**
2.1 The model . . . 3
2.2 Assumptions . . . 3
2.3 The three tasks . . . 3
2.4 Notation . . . 4
2.5 Running examples . . . 4

3 Background: linear algebra, Gaussian algebra and Bayesian updating **4**
3.1 Linear algebra . . . 4
3.2 Gaussian algebra . . . 5
3.3 Bayesian updating . . . 5

4 Methods **6**
4.1 The Kalman filter . . . 6
4.2 Prediction . . . 6
4.3 The RTS smoother . . . 7
4.4 Theoretical support . . . 7
4.5 Computational cost . . . 7
4.6 Worked examples . . . 8

5 Parameter estimation **8**
5.1 Latent variable models . . . 8
5.2 The idea: a simpler problem in more variables . . . 8
5.3 The variational bound and the EM algorithm . . . 9
5.4 Example: EM for a Gaussian mixture . . . 10
5.5 EM for linear-Gaussian models . . . 10
5.6 Misspecification and filter stability . . . 11

6 Implementation and numerics **11**
6.1 Design of the codebase . . . 11
6.2 Numerical variants . . . 11
6.3 Validation . . . 12
6.4 Simulation study . . . 12

7 Individual extensions **12**
7.1 Pairs trading with state space models (Aseem Kulkarni) . . . 12
7.2 Interacting particle systems (Victor Sze) . . . 12
7.3 Tracking footballers' market values (Nigel Prem Lim) . . . 14
7.4 Time-varying market beta (Lozko Pilev) . . . 14

8 Conclusion **16**

Division of work **17**

1 Introduction and aims

To write here.

- Open with the question, not the algorithm: a system evolves, we see it only indirectly, and we want to know what is going on inside it. Separate the two things we might want: what was the state, and what were the parameters.
- Give real-world settings where data has this shape: navigation and tracking, weather prediction, sports player and team ratings, epidemic modelling, econometrics and finance. One sentence each; say what is hidden and what is measured in each.
- Explain the pipeline the report follows: phenomenon → mathematical model → tasks (filtering, smoothing, parameter estimation) → algorithms that solve those tasks.
- Close with the aims and an outline of the sections. Cite the two main texts here, [5] and [4].

1.1 Why linear-Gaussian models?

To write here.

- General state space models are hard; narrowing to a class where the tasks are tractable is what makes an exact treatment possible. Mention the other common tractable choice, finite state spaces, in a sentence, to show the linear-Gaussian choice is a choice.
- Three reasons: it is *possible* (everything is exact and in closed form), it is *reasonable* (many systems really are approximately linear with roughly Gaussian noise), and it is often a *good approximation* to a smooth nonlinear system.
- Say honestly what this does not give you: it does not make all state space models easy, but it is the case worth understanding first.

The third reason is linearisation: a smooth nonlinear system behaves locally like a linear-Gaussian one,

$$x_t = f(x_{t-1}) + \sigma\xi_t \quad \approx \quad f(\mu) + f'(\mu)\,(x_{t-1} - \mu) + \sigma\xi_t. \tag{1}$$

To write here.

- Explain (1) in words and say where the approximation breaks down.

2 Model and tasks

2.1 The model

The hidden state is $x_t \in \mathbb{R}^d$ and the measurement is $y_t \in \mathbb{R}^k$, for $t = 1, \dots, T$:

$$\text{Dynamics:} \qquad x_t = F x_{t-1} + q_t, \qquad q_t \sim \mathcal{N}(0, Q), \tag{2}$$

$$\text{Measurement:} \qquad y_t = H x_t + r_t, \qquad r_t \sim \mathcal{N}(0, R), \tag{3}$$

$$\text{Prior:} \qquad x_0 \sim \mathcal{N}(m_0, P_0). \tag{4}$$

Equivalently, in conditional form,

$$x_t \mid x_{t-1} \sim \mathcal{N}(F x_{t-1},\, Q), \qquad y_t \mid x_t \sim \mathcal{N}(H x_t,\, R). \tag{5}$$

To write here.

- Give the shape of every matrix and say it out loud: F is $d \times d$, Q is $d \times d$ and positive semi-definite, H is $k \times d$, R is $k \times k$ and positive definite. Checking shapes is the quickest way to catch an error in what follows.
- Say in words what each equation means: the state moves, and we see a noisy linear function of it.
- Note that (2)–(4) and (5) are the same model written two ways, and that both appear in the literature.

A known constant drift u is a harmless variant, used by the running example of Section 2.5; it changes the prediction mean from Fm to $Fm + u$ and nothing else:

$$x_t = F x_{t-1} + u + q_t. \tag{6}$$

2.2 Assumptions

To write here.

- State the four assumptions and, for each, one sentence on what it buys us and one on when it might fail in practice.
- (A1) Markov dynamics: x_t depends on the past only through x_{t-1}.
- (A2) The measurements are conditionally independent given the current state.
- (A3) q_t, r_t and x_0 are mutually independent and Gaussian.
- (A4) F, Q, H, R are known. Say explicitly that Section 5 drops this one.

2.3 The three tasks

To write here.

- Introduce the three questions and keep the names straight, since the rest of the report uses them constantly. The distinction is only which measurements we are allowed to condition on.
- Say which task each later section answers: filtering in Section 4.1, prediction in Section 4.2, smoothing in Section 4.3.

$$\text{Filtering:} \quad p(x_t \mid y_{1:t}) \quad \text{the state now,} \tag{7}$$

$$\text{Prediction:} \quad p(x_{t+h} \mid y_{1:t}) \quad \text{the state in the future,} \tag{8}$$

$$\text{Smoothing:} \quad p(x_t \mid y_{1:T}) \quad \text{a past state, using all the data.} \tag{9}$$

2.4 Notation

To write here.

- Fix the notation here, once, and make everybody use it. The moments carry two indices: the time of the state, and the last measurement conditioned on.
- $m_{t|t-1}, P_{t|t-1}$ are the *predictive* moments; $m_{t|t}, P_{t|t}$ the *filtering* moments; $m_{t|T}, P_{t|T}$ the *smoothing* moments.
- Measurements run $y_1, \dots, y_T$, and the prior sits at $t = 0$ with no measurement attached to it. The state sequence therefore runs $x_0, \dots, x_T$, one longer than the measurements.
- Warn the reader about one clash: the general latent-variable literature, and Section 5 where it is quoted, often writes z for the latent variable and x for the observed data. This report keeps x for the state and y for the measurement throughout. Say so once, here.

2.5 Running examples

To write here.

- Introduce the examples once here and reuse them throughout, so the reader meets no new model in the results section.
- A two-dimensional drifting object: state is position, the drift is known, both coordinates are observed with noise. This is the one to carry through the whole report.
- Vehicle tracking: state is position and velocity, only position is observed. Good for showing what smoothing adds, since velocity is never measured directly.
- A drifting regression coefficient, such as a hedge ratio or a market beta: the state is the coefficient and the measurement matrix H_t holds the regressor, so this is the first example where H depends on t. Two of the individual extensions use it.

3 Background: linear algebra, Gaussian algebra and Bayesian updating

To write here.

- This section exists so the report is self-contained. Keep it brisk: these are tools, not results.

3.1 Linear algebra

To write here.

- Block matrices and the Schur complement, since conditioning a Gaussian is a Schur complement in disguise.
- The matrix inversion lemma (Woodbury), and where it is used: rewriting the update in information form, and explaining why the $k \times k$ inverse is the one worth doing when $k \ll d$.
- State both, prove neither, cite [5, App. A.2].

$$M = \begin{pmatrix} A & B \\ C & D \end{pmatrix}, \qquad M/D := A - BD^{-1}C \quad \text{(Schur complement of } D\text{)}. \tag{10}$$

$$(A + UCV)^{-1} = A^{-1} - A^{-1}U\left(C^{-1} + VA^{-1}U\right)^{-1}VA^{-1}. \tag{11}$$

3.2 Gaussian algebra

To write here.

- These three lemmas are the whole content of the Kalman filter. Everything in Section 4 is an application of them, so state them carefully and prove the first two in full.
- After the three, say the consequence out loud: every filtering and smoothing distribution stays Gaussian, so it is enough to track a mean and a covariance. That sentence is why the algorithm exists.

Lemma 3.1 (Linear propagation)**.** *If $x \sim \mathcal{N}(m, P)$ and $x' \mid x \sim \mathcal{N}(Ax, B)$, then*

$$x' \sim \mathcal{N}\Big(Am,\ APA^\top + B\Big)\,. \tag{12}$$

To write here.

- Prove this in three lines with the laws of total expectation and total variance: $\mathbb{E}[x'] = \mathbb{E}[\mathbb{E}[x' \mid x]] = \mathbb{E}[Ax] = Am$, and $\mathrm{Cov}[x'] = \mathrm{Cov}[\mathbb{E}[x' \mid x]] + \mathbb{E}[\mathrm{Cov}[x' \mid x]] = A\,\mathrm{Cov}[x]A^\top + B$.
- Note the small fact $\mathrm{Cov}[Ax] = A\,\mathrm{Cov}[x]A^\top$ that it leans on.
- Say what it gives us: a Gaussian prior plus linear-Gaussian dynamics gives a Gaussian predictive distribution. This is the prediction step.

Lemma 3.2 (Joint Gaussianity)**.** *If $x \sim \mathcal{N}(m, P)$ and $y \mid x \sim \mathcal{N}(Hx, R)$, then (x, y) is jointly Gaussian with*

$$\begin{pmatrix} x \\ y \end{pmatrix} \sim \mathcal{N}\left(\begin{pmatrix} m \\ Hm \end{pmatrix},\ \begin{pmatrix} P & PH^\top \\ HP & HPH^\top + R \end{pmatrix}\right). \tag{13}$$

To write here.

- Prove it by computing the two cross-covariance blocks; it is a good linear algebra exercise and takes a few lines.
- Check the shapes of all four blocks as you go.

Lemma 3.3 (Conditioning)**.** *If (x, y) is jointly Gaussian with means m_x, m_y and covariance blocks P_{xx}, P_{xy}, P_{yy}, then*

$$x \mid y \sim \mathcal{N}\big(m_x + P_{xy}P_{yy}^{-1}(y - m_y),\ P_{xx} - P_{xy}P_{yy}^{-1}P_{yx}\big)\,. \tag{14}$$

To write here.

- State it, cite [5, App. A.1] for the proof, and point out that the posterior covariance is the Schur complement (10).
- Explain the mean in words: we correct the prior mean by the *discrepancy between the predicted measurement and the one we actually saw*, scaled by how much the two variables covary relative to how noisy y is. This sentence is what makes the Kalman gain intuitive later, so it is worth getting right here.

3.3 Bayesian updating

To write here.

- Describe the recursion in words before any algebra: a belief about the state is pushed through the dynamics to give a predictive belief, then conditioned on the new measurement by Bayes' rule to give the next belief, and the cycle repeats.
- A figure would earn its place here: uncertainty growing between measurements and shrinking at each one.
- Make the point that the filtering problem is *recursive* – nothing needs re-reading, the whole past enters through $m_{t-1|t-1}, P_{t-1|t-1}$.

$$\underbrace{p(x_{t-1} \mid y_{1:t-1})}_{\text{filter}} \xrightarrow[\text{Lemma 3.1}]{\text{dynamics}} \underbrace{p(x_t \mid y_{1:t-1})}_{\text{predictive}} \xrightarrow[\text{Lemma 3.3}]{\text{data, Bayes}} \underbrace{p(x_t \mid y_{1:t})}_{\text{filter}} \tag{15}$$

4 Methods

4.1 The Kalman filter

For $t = 1, \dots, T$, the prediction step is

$$m_{t|t-1} = F\, m_{t-1|t-1}, \qquad P_{t|t-1} = F\, P_{t-1|t-1} F^\top + Q, \tag{16}$$

and the update step, on seeing y_t, is

$$v_t = y_t - H\, m_{t|t-1}, \qquad S_t = H\, P_{t|t-1} H^\top + R, \tag{17}$$

$$K_t = P_{t|t-1} H^\top S_t^{-1}, \tag{18}$$

$$m_{t|t} = m_{t|t-1} + K_t v_t, \qquad P_{t|t} = (I - K_t H)\, P_{t|t-1}. \tag{19}$$

To write here.

- Derive, do not assert. (16) is Lemma 3.1 with $A = F$, $B = Q$. For (17)–(19), apply Lemma 3.2 to the predictive distribution and then Lemma 3.3, and read off the blocks.
- Name the pieces: v_t the *innovation*, S_t its covariance, K_t the *Kalman gain* [2].
- Explain what the gain does: when the measurement is noisy (R large) the gain is small and we barely move; when the state is uncertain ($P_{t|t-1}$ large) the gain is large and we move a lot. Check the two extreme cases explicitly, it is worth the lines.
- Say that the filter is *online*: each step uses only the previous moments and the new measurement.
- Note that the same formulas hold verbatim for time-varying F_t, Q_t, H_t, R_t – same algebra, more subscripts.

Algorithm 1 The Kalman filter, as two alternating maps

1: $(m_{0|0}, P_{0|0}) \leftarrow (m_0, P_0)$
2: **for** $t = 1, \dots, T$ **do**
3: $\quad (m_{t|t-1}, P_{t|t-1}) \leftarrow \textsc{Propagate}(m_{t-1|t-1}, P_{t-1|t-1})$ ▷ push through the dynamics
4: $\quad (m_{t|t}, P_{t|t}) \leftarrow \textsc{Assimilate}_{y_t}(m_{t|t-1}, P_{t|t-1})$ ▷ condition on the measurement
5: **end for**
6: **return** $\{(m_{t|t}, P_{t|t})\}_{t=0}^{T}$

To write here.

- Define the two maps of Algorithm 1 explicitly: $\textsc{Propagate} : (m, P) \mapsto (Fm,\, FPF^\top + Q)$ and $\textsc{Assimilate}_y : (m, P) \mapsto (m + K(y - Hm),\, (I - KH)P)$ with $K = PH^\top (HPH^\top + R)^{-1}$.
- Splitting it this way is worth doing because the same two maps reappear in the smoother and in the E-step of Section 5.

4.2 Prediction

Prediction needs no new algebra: with no measurement to assimilate, the filter is simply propagated forward. For $h \geq 1$,

$$m_{t+h|t} = F\, m_{t+h-1|t}, \qquad P_{t+h|t} = F\, P_{t+h-1|t} F^\top + Q, \tag{20}$$

started from the filtering moments $(m_{t|t}, P_{t|t})$.

To write here.

- Keep this short – it is a corollary, not a new method – but do not omit it, since Section 2.3 promised three tasks and this is the third.
- Say what happens as h grows: the mean relaxes towards the stationary behaviour of F and the covariance grows towards the stationary covariance, if one exists. Connect this to the picture in Section 3.3, where uncertainty grows between measurements.
- Note the equivalent implementation view: predicting h steps is the same as filtering h missing measurements.

4.3 The RTS smoother

Starting from $m_{T|T}$ and $P_{T|T}$, for $t = T-1, \dots, 1, 0$:

$$G_t = P_{t|t} F^\top \left(P_{t+1|t}\right)^{-1}, \tag{21}$$

$$m_{t|T} = m_{t|t} + G_t \left(m_{t+1|T} - m_{t+1|t}\right), \tag{22}$$

$$P_{t|T} = P_{t|t} + G_t \left(P_{t+1|T} - P_{t+1|t}\right) G_t^\top. \tag{23}$$

The lag-one posterior cross-covariances, needed in Section 5, come from the same gains:

$$P_{t,t+1|T} := \mathrm{Cov}(x_t, x_{t+1} \mid y_{1:T}) = G_t \, P_{t+1|T}. \tag{24}$$

To write here.

- Note the range: the recursion runs down to $t = 0$, not to $t = 1$. The last step smooths the prior itself, giving $m_{0|T}, P_{0|T}$, and those are needed by (42). Running to $t = 0$ also makes (24) produce exactly T cross-covariances, one for each pair (x_{t-1}, x_t) with $t = 1, \dots, T$, which is what the M-step sums over.
- Derive (21)–(23) from the joint distribution of (x_t, x_{t+1}) given $y_{1:t}$, conditioning on x_{t+1} and then averaging over the smoothed x_{t+1}. The Markov property is what lets you condition on $y_{1:t}$ rather than $y_{1:T}$ in the middle step – say so explicitly, it is the one subtle point in the derivation. Cite [3].
- Explain the shape of (22): the correction is the amount by which the future turned out to differ from what was predicted, pushed back through G_t.
- Note that the smoother reads only the filter's output, so one forward pass serves both.
- Check the two sanity properties and say why they must hold: at $t = T$ the smoother equals the filter, and $P_{t|T} \preceq P_{t|t}$ – more data cannot widen the posterior.

4.4 Theoretical support

To write here.

- State each claim precisely, then either prove it or cite it. Say which you did; a report that blurs the two is weaker than one that admits a citation.
- *Exactness*: the recursion returns the true conditional distribution, not an approximation to it.
- *Batch equivalence*: the filtering distribution at time t equals the posterior obtained by conditioning the whole joint Gaussian of $(x_{1:t}, y_{1:t})$ in one step. This is both a theorem and the basis of the numerical test in Section 6.
- *Optimality*: the filtering mean is the minimum mean-squared-error estimator of x_t given $y_{1:t}$.
- *Stability*: the effect of a wrong prior washes out as t grows. State the conditions, cite [4, Ch. 7–8].
- *Likelihood*: the filter evaluates the likelihood exactly, for free, by the prediction-error decomposition below. This is the bridge to Section 5.

$$\log p(y_{1:T}) = \sum_{t=1}^{T} \log p(y_t \mid y_{1:t-1}) = -\tfrac{1}{2} \sum_{t=1}^{T} \left[k \log 2\pi + \log|S_t| + v_t^\top S_t^{-1} v_t \right]. \tag{25}$$

4.5 Computational cost

To write here.

- Count the cost of one step and multiply: the expensive operations are a $d \times d$ product and a $k \times k$ solve.
- Contrast with the batch solution, which inverts a $Td \times Td$ matrix. The gap is the point of the whole algorithm, so state it as a proposition and confirm it with timings in Section 6.
- Say what is linear in T and what is not.

$$\underbrace{O\left(d^3 + k^3\right) \text{ per step}}_{\text{Kalman filter}} \qquad \text{versus} \qquad \underbrace{O\left((Td)^3\right)}_{\text{batch solution}}. \tag{26}$$

4.6 Worked examples

To write here.

- Start with the scalar case $d = k = 1$, where every quantity can be written out by hand. Do one step symbolically, so the reader sees the gain as a ratio of variances.
- Then the two-dimensional running example, with a figure: the true path, the measurements, the filtered mean and a credible band.
- Then a figure comparing filtering with smoothing on the same run, which is the clearest way to show what the backward pass buys.

Example 4.1 (The scalar filter)**. To write here.**

- Take $F = H = 1$, $Q = q$, $R = r$, $P_0 = p_0$, and write out one step. The gain becomes $K = (p_0 + q)/(p_0 + q + r)$: a ratio of how uncertain we are to how uncertain everything is.
- Read off the two limits, $r \to 0$ and $r \to \infty$, and say what each means.

5 Parameter estimation

To write here.

- Open by dropping assumption (A4). Until now F, Q, H, R were given; in an application they are not, and the question becomes whether we can learn them from the data, and how.
- Say which of the two questions this answers: not "what was the state", but "what were the parameters".
- Warn about notation once more: the general theory below is usually written with a latent z and observed data; here the latent object is the whole state path $x_{0:T}$ and the data is $y_{1:T}$.

5.1 Latent variable models

To write here.

- Set up the general problem before the state space one, because the argument is general and the state space case is one instance.
- We have a parameter θ, a latent variable we model but do not observe, observed data, and a tractable joint p_θ.
- State the premise that makes everything work: *if we could see the latent variable too, fitting θ would be easy* – and not just the plain MLE but a *weighted* one, with any non-negative weights. Keep that word, the weights are what appears in the M-step.
- The obstacle is the marginal: the integral over all latent paths.

$$\max_{\theta \in \Theta} \log p_\theta(y_{1:T}) = \log \left(\int p_\theta(x_{0:T}, y_{1:T}) \, dx_{0:T} \right). \tag{27}$$

To write here.

- Then note that for an LGSSM we are luckier than in general: (25) evaluates that integral exactly, so direct numerical maximisation of the likelihood is available. Mention it as the baseline against which EM should be compared.

5.2 The idea: a simpler problem in more variables

To write here.

- Give the idea in its plainest form first, with no probability in sight. It is an optimisation trick, and EM is one instance of it.

- Suppose we want $\min_x F(x)$ with F complicated, and suppose F can be written as $F(x) = \min_y G(x, y)$ with G simple. Then minimising F over x is minimising G over (x, y): a harder problem in more variables that is easier because G is nicer.
- Then alternate: fix y and minimise over x, fix x and minimise over y. Neither half ever increases G.

$$F(x) = \min_y G(x, y) \qquad \Longrightarrow \qquad \min_x F(x) = \min_{x,\, y} G(x, y). \tag{28}$$

Example 5.1 (The lasso)**.** The lasso objective $\|Y - X\beta\|^2 + \lambda\|\beta\|_1$ is awkward because $\|\beta\|_1$ is not quadratic. But for any $\beta \in \mathbb{R}$,

$$|\beta| = \min_{t>0} \left\{ \frac{\beta^2}{2t} + \frac{t}{2} \right\}, \tag{29}$$

so the problem becomes

$$\min_{\beta \in \mathbb{R}^p} \min_{t \in \mathbb{R}^p_{>0}} \left\{ \|Y - X\beta\|^2 + \frac{\lambda}{2} \sum_{j=1}^{p} \left(\frac{\beta_j^2}{t_j} + t_j \right) \right\}. \tag{30}$$

To write here.

- Prove (29) by differentiating in t: the optimum is $t = |\beta|$, which gives $|\beta|/2 + |\beta|/2$.
- Then make the point of the example: for fixed t, (30) is ridge regression, solvable in closed form; for fixed β, optimising t is immediate. A problem with no closed-form solution has become two that do.
- Say why this example is here: it is (28) with no probability anywhere, and the same move produces EM below. It also returns in Section 7.2, where a sparsity penalty is placed on an interaction network.

5.3 The variational bound and the EM algorithm

For any density q over the latent path, Jensen's inequality applied to the concave function log gives

$$\log p_\theta(y_{1:T}) = \log \int q(x_{0:T}) \frac{p_\theta(x_{0:T}, y_{1:T})}{q(x_{0:T})} dx_{0:T} \;\geq\; \int q \log \frac{p_\theta(x_{0:T}, y_{1:T})}{q} =: \mathcal{F}(\theta, q), \tag{31}$$

with equality exactly when q is the posterior $p_\theta(x_{0:T} \mid y_{1:T})$, so that

$$\max_\theta \log p_\theta(y_{1:T}) = \max_{\theta,\, q} \mathcal{F}(\theta, q). \tag{32}$$

EM is alternating maximisation of $\mathcal{F}$: until convergence,

$$\text{E-step:} \qquad q_j = p_{\theta_{j-1}}(x_{0:T} \mid y_{1:T}), \tag{33}$$

$$\text{M-step:} \qquad \theta_j = \arg\max_{\theta \in \Theta} \int q_j(x_{0:T}) \log p_\theta(x_{0:T}, y_{1:T})\, dx_{0:T}. \tag{34}$$

To write here.

- Explain the trick in (31): multiply and divide by q, recognise the result as an expectation over $x \sim q$, then apply $\log \mathbb{E}[\,\cdot\,] \geq \mathbb{E}[\log \cdot\,]$, with equality iff the argument is constant.
- Point out that (32) is exactly (28): a complicated function of few variables has become a simpler function of more.
- Say where the names come from: (33) is an *expectation*, (34) a *maximisation*.
- Say why each half is easy: given θ, the optimal q is known outright, it is the posterior; given q, the objective is a *weighted joint maximum likelihood* problem, which is the thing assumed tractable in Section 5.1.
- Remark, as is worth remarking, that it is *not* obvious a priori that this works. Then give the argument: $\mathcal{F}$ cannot decrease at either half-step, so the observed log-likelihood never decreases. State clearly what this does and does not promise – monotone, not globally convergent, and the limit can be a local maximum.

5.4 Example: EM for a Gaussian mixture

To write here.

- Put a small, fully explicit example before the state space algebra, so the reader has seen EM work once already.

Take K clusters, a latent label $z \in \{1, \dots, K\}$ drawn uniformly, and cluster centres $\theta = (\mu_1, \dots, \mu_K)$, with

$$p_\theta(z, x) = \tfrac{1}{K}\, \mathcal{N}(x; \mu_z, I). \tag{35}$$

Fixing θ, the E-step is a ratio of densities, and fixing q, the M-step is a weighted average:

$$q_{ik} = \frac{\mathcal{N}(x_i; \mu_k, I)}{\sum_{l=1}^{K} \mathcal{N}(x_i; \mu_l, I)}, \qquad \hat{\mu}_k = \frac{\sum_{i=1}^{N} q_{ik}\, x_i}{\sum_{i=1}^{N} q_{ik}}. \tag{36}$$

To write here.

- Derive (36): the M-step objective is $\sum_i \sum_k q_{ik} \log \mathcal{N}(x_i; \mu_k, I)$, which is $-\tfrac{1}{2} \sum_i \sum_k q_{ik} \|x_i - \mu_k\|^2$ up to constants, and setting the derivative to zero gives the weighted mean.
- Say what q_{ik} means in words – how strongly point i is attributed to cluster k – and note that these are exactly the non-negative weights promised in Section 5.1.
- One sentence on the relationship to k-means, if you want it: k-means is what you get by forcing each q_{ik} to be 0 or 1.

5.5 EM for linear-Gaussian models

The complete-data log-likelihood factorises along the model:

$$\log p_\theta(x_{0:T}, y_{1:T}) = \log \mathcal{N}(x_0; m_0, P_0) + \sum_{t=1}^{T} \log \mathcal{N}(x_t; F x_{t-1}, Q) + \sum_{t=1}^{T} \log \mathcal{N}(y_t; H x_t, R). \tag{37}$$

Taking its expectation under q_j leaves it depending on the smoothed moments only:

$$L_j(\theta) = \mathcal{F}\big(\theta,\ \{m_{t|T}\},\ \{P_{t|T}\},\ \{P_{t,t+1|T}\}\big)\,, \tag{38}$$

so the E-step is a Kalman filter followed by an RTS smoother. Writing

$$S_{11} = \sum_{t=1}^{T} \Big(P_{t|T} + m_{t|T} m_{t|T}^\top\Big)\,, \quad S_{10} = \sum_{t=1}^{T} \Big(P_{t-1,t|T}^\top + m_{t|T} m_{t-1|T}^\top\Big)\,, \quad S_{00} = \sum_{t=1}^{T} \Big(P_{t-1|T} + m_{t-1|T} m_{t-1|T}^\top\Big)\,, \tag{39}$$

the M-step is available in closed form:

$$F^* = S_{10} S_{00}^{-1}, \qquad Q^* = \tfrac{1}{T}\Big(S_{11} - F^* S_{10}^\top\Big)\,, \tag{40}$$

$$H^* = \left(\sum_{t=1}^{T} y_t m_{t|T}^\top\right) S_{11}^{-1}, \quad R^* = \tfrac{1}{T} \sum_{t=1}^{T} \Big[\big(y_t - H^* m_{t|T}\big)\big(y_t - H^* m_{t|T}\big)^\top + H^* P_{t|T} H^{*\top}\Big]\,, \tag{41}$$

$$m_0^* = m_{0|T}, \qquad P_0^* = P_{0|T}. \tag{42}$$

To write here.

- Derive (39): take the expectation of (37) and collect terms. Explain why the lag-one cross-covariances appear – the dynamics term contains the product $x_t x_{t-1}^\top$, and nothing else in the report produces it.
- Make the key point explicitly, because it is the one thing a reader is likely to get wrong: where a state appears *linearly* it is replaced by its smoothed mean, but where it appears *quadratically* it is replaced by the second moment, mean outer mean *plus* the smoothed covariance. Dropping that covariance gives an algorithm that still runs and still looks plausible, and is wrong.

- Derive one of (40) in full as a weighted least squares problem, and say the others follow the same way.
- Say what is general and what is not: the E-step is the same machinery whatever we are estimating, while the M-step depends on how much of (F, Q, H, R) is known and how much is being learned. Write down the updates only for the parameters actually estimated, and carry the rest through fixed.
- Draw the loop these equations close: filtering feeds parameter estimation, which feeds smoothing, which feeds filtering. A small figure of that cycle would be worth including.
- Mention identifiability: F and H are determined only up to a similarity transform, since (F, H, x) and (AFA^{-1}, HA^{-1}, Ax) give the same law for y. So fix one of them in practice.

5.6 Misspecification and filter stability

To write here.

- Ask the practical question: what if the parameters we filter with are not the ones that generated the data?
- Say how it will be studied – an experiment, not only a theorem: simulate under one model, filter under another, and measure the damage with the diagnostics of Section 6.
- Note which diagnostics notice misspecification and which do not. Point estimates can look reasonable while the error bars are badly wrong, which is exactly what NEES detects and RMSE does not.

The quantitative form of the question is whether a small error in the parameters implies a small error in the filtered state:

$$\|F - F_*\| \le \varepsilon \quad \stackrel{?}{\Longrightarrow} \quad \|m_t - m_{t,*}\| \le \varepsilon'. \tag{43}$$

6 Implementation and numerics

6.1 Design of the codebase

To write here.

- Describe what was built and the one design decision behind it: rather than code each example separately, implement *any* LGSSM once, and let every example and extension call it.
- The interface: given $(F, Q, H, R, m_0, P_0, y_{1:T})$, with the constraints checked, return the filtering and smoothing distributions and the likelihood.
- Say what the code handles beyond the basic case: time-varying parameters, missing measurements, a known drift.
- Give the repository URL and the commit the reported results come from, so the numbers can be reproduced.
- Keep this short and factual. The reader wants to know what it does, not to read the source.

6.2 Numerical variants

To write here.

- Explain the problem first: in exact arithmetic (19) is fine, but in floating point the covariance can lose symmetry and eventually positive definiteness, and a negative variance makes the run meaningless.
- Then the Joseph form below: algebraically identical, but a sum of two terms of the form $MM^\top$, so positive semi-definite by construction. Say what it costs.
- Then the square-root filter in a sentence or two: propagate a factor C with $P = CC^\top$, so P cannot fail to be positive semi-definite. Cite rather than derive.
- Say which is the default in the code and why.

$$P_{t|t} = (I - K_t H) P_{t|t-1} (I - K_t H)^\top + K_t R K_t^\top \qquad \text{(Joseph form)}. \tag{44}$$

6.3 Validation

To write here.

- Present the tests as evidence for the claims of Section 4, in the same order, and say which claim each one supports.
- **Exactness.** Compare against the batch posterior, which is computed by conditioning one dense joint Gaussian and shares no code with the recursion. Report the agreement. Make the point that testing a recursion against another recursion would prove much less.
- **Accuracy.** RMSE of the filter and the smoother against the true simulated state, on the running examples. Expect the smoother to win, and say why.
- **Calibration.** Coverage of credible intervals, and NEES. Define both, and give the acceptance interval for the NEES test, citing [1].
- **Scalability and robustness.** Run time as d and k grow, against the predicted scaling (26); then missing measurements and time-varying models.

$$\epsilon_t = \left(x_t - m_{t|t}\right)^\top P_{t|t}^{-1} \left(x_t - m_{t|t}\right) \;\sim\; \chi^2_d, \qquad v_t^\top S_t^{-1} v_t \;\sim\; \chi^2_k. \tag{45}$$

To write here.

- Explain the difference between the two quantities in (45) and when each is usable: the first needs the true state, so it exists only in simulation; the second needs only the measurements, so it is the check available on real data. Every extension that touches real data will need the second one.
- A filter that reports ϵ_t much larger than d is overconfident: its covariances are too small for the errors it is actually making.

6.4 Simulation study

To write here.

- Collect the figures and tables here, each with a sentence saying what it shows.
- Suggested: true path with filtered and smoothed estimates and their bands; filtering versus smoothing uncertainty over time; a run-time plot against d with the predicted slope; a NEES plot with its acceptance interval, for a correct and a misspecified filter; and the log-likelihood increasing along the EM iterations.

7 Individual extensions

To write here.

- Each author writes their own subsection. Keep a common shape: model, data, what was done, what was found.

7.1 Pairs trading with state space models (Aseem Kulkarni)

7.2 Interacting particle systems (Victor Sze)

Each of N agents has its own state $x_t^i \in \mathbb{R}^d$, and the agents influence one another through an *interaction network* B:

$$x_t^i = A_i\, x_{t-1}^i + \sum_{j \neq i} B_{ij}\, x_{t-1}^j + q_t^i, \qquad i = 1, \ldots, N. \tag{46}$$

Stacking the agents into $x_t = (x_t^1, \dots, x_t^N) \in \mathbb{R}^{Nd}$ turns (46) into a single LGSSM of dimension Nd,

$$x_t = \mathbf{M}\, x_{t-1} + q_t, \qquad \mathbf{M} = \begin{pmatrix} A_1 & B_{12} & \cdots & B_{1N} \\ B_{21} & A_2 & \cdots & B_{2N} \\ \vdots & & \ddots & \vdots \\ B_{N1} & B_{N2} & \cdots & A_N \end{pmatrix}, \tag{47}$$

so the methods of Section 4 apply unchanged.

To write here.

- Say what B_{ij} *means*: the strength with which agent j influences agent i at the next step. $B_{ij} = 0$ says agent j has no direct effect on agent i. The matrix is not symmetric in general – influence can run one way.
- Note the cost problem this creates: the stacked model has dimension Nd, so a general Kalman filter step costs $O\big((Nd)^3\big)$, cubic in the number of agents.

Mean-field coupling, and the decoupled filter. A special case is coupling through the group average $\bar{x}_{t-1} = \frac{1}{N}\sum_j x_{t-1}^j$, that is $A_i = A$ and $B_{ij} = B/N$ for every pair:

$$x_t^i = A\, x_{t-1}^i + B\, \bar{x}_{t-1} + q_t^i. \tag{48}$$

To write here.

- Show that changing coordinates to the group average and the deviations from it block-diagonalises $\mathbf{M}$, so the Nd-dimensional filter splits into independent d-dimensional filters. State the result as a proposition and prove it.
- Then state the payoff precisely: the same exact answer at a cost linear in N rather than cubic. Confirm it with timings, reusing the scaling experiment of Section 6.
- Be clear that this is an exact reformulation, not an approximation – that is what makes it worth doing.

Learning the interaction network. When A and B are not known, they are parameters to be estimated, and Section 5 applies with $\theta = (A, B)$. Two levels:

To write here.

- **Simple starting point: fix the network a priori.** Set $B_{ij} = 0$ for pairs that cannot plausibly interact – agents far apart, or in unrelated roles. This is dimension reduction decided in advance: fewer parameters to estimate and a sparser $\mathbf{M}$. Say how you chose which entries to zero, and admit that the choice is an assumption.
- **More advanced: learn the network from the data.** Estimate (A, B) by maximum likelihood with a penalty on $\{B_{ij}\}$ that encourages sparsity, so the data decides which interactions exist rather than you. Write the objective as below.
- Connect the two sections: the penalised M-step is Example 5.1 again – a sparsity penalty handled by the same alternating argument that produced EM in the first place. That link is worth making explicitly, because it is the reason both appear in this report.
- Say what you will compare: the a priori network against the learned one, on recovery of the true B in simulation and on fit in the application.

$$\hat{\theta} = \arg\max_{\theta=(A,B)} \left\{ \log p_\theta(y_{1:T}) - \lambda \sum_{i \neq j} \|B_{ij}\|_1 \right\}. \tag{49}$$

Application. To write here.

- Name the dataset here, and state what an agent is, what the state x_t^i is, and what is actually measured. Two candidates, to be settled:
- **Football.** Agents are the 11 players of a team, x_t^i is position and velocity on the pitch, and the measurement is the tracking data. The a priori network comes from playing position: set $B_{ij} = 0$ for players who are far apart or in unrelated roles. Note that this is a different dataset and question from Section 7.3, which values players rather than tracking them.
- **Traffic.** Agents are vehicles or road sensors, x_t^i is speed or flow, and the measurement is the loop-detector or probe data. The a priori network comes from the road topology: only adjacent segments interact directly.
- Whichever is chosen, say early what is hidden and what is observed, and check the data is actually obtainable before building anything on it.

7.3 Tracking footballers' market values (Nigel Prem Lim)

7.4 Time-varying market beta (Lozko Pilev)

A stock's sensitivity to movements in the overall market is measured by its market beta. Traditional regression models assume that beta remains constant over time. However, a stock's exposure to market risk may change due to economic conditions, company performance, and market volatility.

We model beta as a hidden state that evolves over time within a linear-Gaussian state space model.

The time-varying beta model. Let β_t represent the market beta of a stock at time t. The state and observation equations are

$$\beta_t = \beta_{t-1} + q_t, \qquad q_t \sim \mathcal{N}(0, Q), \tag{50}$$

$$y_t = H_t \beta_t + r_t, \qquad r_t \sim \mathcal{N}(0, R). \tag{51}$$

Here y_t is the observed excess return of the stock, H_t is the observed excess return of the market index, Q is the process noise variance, and R is the observation noise variance. We assume independent Gaussian noises and the prior $\beta_0 \sim \mathcal{N}(m_0, P_0)$.

This is a special case of the model in Section 2, with $F = 1$ and a time-varying observation matrix H_t. The model assumes a zero-intercept regression.

To write here.

- Explain what market beta represents and why it may change over time.
- Explain the interpretation of the hidden state β_t and observed returns y_t and H_t.
- Discuss how Q determines the flexibility of beta and how R represents observation noise.
- Explain why the Kalman filter is suitable for estimating a time-varying regression coefficient.

Estimating the unknown noise parameters. In financial applications, the process and observation noise variances Q and R are generally unknown. Their values determine how the filter balances changes in the hidden beta against noise in observed returns.

We apply the EM algorithm developed in Section 5 to estimate the parameter vector

$$\theta = (Q, R). \tag{52}$$

For this scalar random-walk model, the general EM updates of Section 5.5 reduce to

$$Q^{\text{new}} = \frac{1}{T}\sum_{t=1}^{T}\Big[(m_{t|T} - m_{t-1|T})^2 + P_{t|T} + P_{t-1|T} - 2P_{t-1,t|T}\Big], \tag{53}$$

$$R^{\text{new}} = \frac{1}{T}\sum_{t=1}^{T}\Big[(y_t - H_t m_{t|T})^2 + H_t^2 P_{t|T}\Big]. \tag{54}$$

The smoothed moments are obtained using the Kalman filter and RTS smoother from Section 4.

To write here.

- Explain how the general EM method is specialised to the time-varying beta model.
- Identify which model parameters are fixed and which are estimated.
- Describe the implementation of the EM updates without repeating the general derivation.
- State the convergence criterion used and how the parameters are initialised.

Synthetic experiments and parameter recovery. We investigate whether EM can recover the unknown noise parameters from simulated financial observations.

Generate a time-varying beta process using known parameters Q_{true} and R_{true}, then estimate these parameters from the observations.

The objective is to investigate whether

$$\widehat{Q} \approx Q_{\text{true}}, \qquad \widehat{R} \approx R_{\text{true}}. \tag{55}$$

To write here.

- Describe how synthetic beta trajectories and stock returns are generated.
- Apply EM to estimate Q and R without using their known true values.
- Compare estimated and true parameters across repeated simulations.
- Investigate how sample size and noise levels affect parameter recovery.
- Examine the convergence of EM under different initial parameter estimates.
- Compare the true beta with filtered and smoothed beta estimates.
- Present parameter recovery errors, convergence plots, and estimation accuracy.

Parameter misspecification and sensitivity. An important question is how the Kalman filter performs when the assumed noise parameters differ from the values that generated the observations.

Suppose the true parameters are $(Q_{\text{true}}, R_{\text{true}})$, while the filter uses $(\widetilde{Q}, \widetilde{R})$.

We investigate how these differences affect the estimated beta and its associated uncertainty.

To write here.

- Generate observations with known values of Q and R.
- Run the filter with correctly specified and deliberately misspecified parameters.
- Study the effects of overestimating and underestimating Q and R separately.

- Compare estimation accuracy using RMSE.
- Evaluate uncertainty calibration using credible interval coverage and NEES, as introduced in Section 6.3.
- Investigate how incorrect parameters affect the responsiveness and stability of beta estimates.
- Examine whether reasonable point estimates can still produce misleading uncertainty intervals.

Application to real financial data. We apply the time-varying beta model to historical stock-market observations.

One possible application is BP against the FTSE 100, using historical price data to calculate stock and market returns.

The noise parameters are estimated using EM, and the Kalman filter is used to obtain a sequence of time-varying beta estimates.

To write here.

- Identify the selected stock, market index, data source, and observation period.
- Calculate stock and market excess returns using appropriately aligned observations.
- Estimate Q and R from the real data.
- Apply the Kalman filter and plot the estimated market beta over time.
- Include uncertainty intervals for the estimated beta.
- Compare the time-varying beta with a constant-beta regression model.
- Assess model fit using standardised innovations, following Section 6.3.
- Discuss whether observed changes in beta have a plausible financial interpretation.

Discussion. To write here.

- Summarise how accurately EM recovered the unknown parameters in simulation.
- Discuss the sensitivity of the Kalman filter to errors in Q and R.
- Evaluate the usefulness of time-varying beta compared with constant-beta estimation.
- Discuss limitations of the model, including Gaussian noise and random-walk dynamics.
- Identify possible improvements and directions for further investigation.

8 Conclusion

To write here.

- What was derived, what was built, what the study showed.
- What the linear-Gaussian assumption bought, and what it cost.
- One paragraph on where this goes next: nonlinear models, high dimensions, approximate filters.

Division of work

To write here.

- State who wrote which sections and who did which parts of the implementation. Sections 1-6 are joint work; Section 7 is individual, one subsection each.
  Section 2: Lozko Pilev
  Section 3: Lozko Pilev
- If the unit requires a declaration of the use of AI tools, this is the place for it.

References

[1] Bar-Shalom, Y., Li, X. R. and Kirubarajan, T. (2001). *Estimation with Applications to Tracking and Navigation*. New York: Wiley.

[2] Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1), 35–45.

[3] Rauch, H. E., Tung, F. and Striebel, C. T. (1965). Maximum likelihood estimates of linear dynamic systems. *AIAA Journal*, 3(8), 1445–1450.

[4] Sanz-Alonso, D., Stuart, A. M. and Taeb, A. (2023). *Inverse Problems and Data Assimilation*. Cambridge: Cambridge University Press.

[5] Särkkä, S. and Svensson, L. (2023). *Bayesian Filtering and Smoothing*, 2nd edn. Cambridge: Cambridge University Press.